
A Same-Generator Corpus and Classifier Floors for AI-Control Monitor Evaluation

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Trusted monitors are tuned and ranked on stored pairs of code. Valid solutions to
2 various programming problems are paired with solutions carrying planted back-
3 doors. However, The two classes are built differently. Honest solutions are human-
4 written, taken from an existing dataset; every backdoor is written by a language
5 model. The label is therefore recoverable from authorship alone, and filtering to
6 working solutions and stripping comments does not remove it. On such a split, a
7 trained bag-of-words classifier scores above roughly four-fifths of the open-weight
8 monitors ranked on it, and a classifier reading no code at all scores above most of
9 them. One model writing both classes removes between a third and nine-tenths of
10 it, on two generators that differ because only one adopts the attack prompt’s layout;
11 matching that prompt removes most of the rest. An LLM monitor is inflated the
12 same way, scoring 0.944 on stored pairs and 0.818 on firing backdoors from the
13 same author. Stored-pair evaluations should report partial-input and bag-of-words
14 floors against permutation nulls, generate both classes from one model and one
15 prompt, and drop inert backdoors.

16 1 Introduction

17 The backbone protocol of AI control is trusted monitoring, where a weaker trusted model reads
18 an untrusted model’s output and flags sabotage [3]. Monitors are developed, tuned and ranked
19 almost entirely on *stored* honest/attack pairs, fixed corpora in which each programming problem
20 contributes an honest solution and a solution carrying a planted backdoor [9, 14, 7]. Honest solutions
21 are human-written, taken from APPS [5]; every backdoor is written by a language model, and the
22 corpus records which one. Separation credited to a monitor may be separation between two authors.

23 There are various known confounding variables between the human and AI solutions, such as in
24 commenting style, so corpora are often preprocessed to mitigate them. The filtered release keeps
25 only working honest solutions and working backdoors and strips every comment [9]; Control Tax
26 reports its pass-tests pre-filter and appendices “Monitoring failed backdoors” [14]; one harness ships
27 a `-leak-check` flag with two trivial baselines in it, a `#-count` detector that falls from 0.6957 to
28 0.4983 once comments go and code length alone at 0.4726 [7]. Out of field, bag-of-words is known
29 to separate human-written from model-written code [11], and the detectors built on it match their
30 classes on line and character count or on shared problem definitions [4, 1]. Adjacent control work
31 attacks other axes, subverting monitors with injected text, showing that attacker-side selection inflates
32 safety estimates, and removing transcript length as a nuisance covariate [15, 2, 12].

33 None of that preprocessing removes the authorship gap. On the filtered, comment-stripped split, a
34 bag-of-words classifier reaches AUROC 0.806, above 20 of the 24 open-weight monitors ranked on
35 that same split, whose median is 0.609; a classifier that reads no code at all, only problem difficulty,
36 test count and question length, reaches 0.772, above 20. The classifier transfers across backdoor



Figure 1: **How a stored pair is built, and how we rebuild it.** Each row is one property of the pair: written once and tied, it is identical on the two sides; written twice, it differs. As stored, the honest program is human-written and the backdoored one model-written, so the classes differ in writer and in elicitation as well as in sabotage. We hold the writer constant, then the prompt as well (the tinted columns), the two prompts being identical apart from the sentences that ask for a flaw. The backdoor row never merges, and it is the only difference a monitor should read. Under each construction is the same bag-of-words classifier’s AUROC on it, on one scale from chance to 1.0. Each is measured on the problems its own construction realises (1,444, 623 and 586 pairs), so the three are a ladder and not a paired comparison, and none of them is the 0.806 floor of §3, which is measured on the split the monitors are ranked on. Every arm’s refitted baseline B' and its ρ are in Table 1.

generators and is nearly as strong on *inert* backdoors as on working ones. Neither is a property of a backdoor.

We contribute (i) a construction for stored-pair corpora, in which one model writes both classes under one prompt framing and one harness verifies both, and the corpus that implements it. Run on two generators, it attributes between a third and nine-tenths of the separability to authorship and most of the remainder to the attack prompt’s framing, and returns a null on the way, since regenerating one half of a stored pair substitutes one confound for another. We add (ii) a second selection artifact the filter installs by defining the negative class through attack failure, and (iii) the construction and reporting practice these imply. Our claims concern monitor training, selection and benchmarking on stored pairs, and our measurement is of one APPS-derived family plus five agentic settings. ControlArena’s live evaluations generate both classes and are not implicated.

2 Setup

We use seven public corpora, whose per-class provenance is the object of study. In the APPS family the honest class is human and the attack class is one of ten named models. Six of the seven declare no licence [8, 13], so we report only aggregates and redistribute no corpus.

Our headline protocol is one of the seven, the filtered, comment-stripped split [9]: 1,209 train problems (2,418 paired items) and 1,375 test items (172 attack, 1,203 honest), labelled by `backdoor_works`, reconstructed with the released harness code [7]. Generated code is scored by our own runner, applied to both classes (appendix A.11).

3 Results

Comment stripping works on comment count and code length, driving both to chance (Table A2, appendix). Two floors on that same split do not move. A monitor scoring at or below a floor has shown nothing beyond what the corpus leaks. Bag-of-words reaches 0.806 and a classifier that reads no code at all reaches 0.772, both far above their permutation nulls and above 20 of the 24 monitors ranked on that split (Figure A1; intervals, nulls and paired-bootstrap counts in appendix A.2).

Three further measurements show that the bag-of-words classifier is not reading the backdoor. It transfers across backdoor generators at 0.969 pairwise: trained on one attacker’s backdoors it identifies another’s, so the feature it has learned is one the attackers share. It is nearly as strong on inert backdoors as on working ones, 0.879 against 0.935, although a backdoor that never fires should be the harder case for a detector of sabotage. The feature it does read is stylistic: `__main__` appears in 0.116 of attack samples against 0.000 of honest ones, a gap the comment stripper leaves untouched and one that replicates on the train split, at 152 against 1 of 1,209. A token-level version of that last test does not replicate (appendix A.12). All three measurements are comment-stripped on both classes; the unstripped values, inflated by the comment channel, are in appendix A.3.

To test whether authorship is what the classifier reads, we hold the author constant and measure how much separability survives. Under the identical pipeline the human-honest baseline is $B = 0.871$ over $n = 1,444$ pairs. Write G for that same quantity when one model writes both classes. Retained signal is $\rho = (G - 0.5)/(B - 0.5)$: $\rho = 1$ means holding authorship constant changed nothing, $\rho = 0$ that nothing survived. We pre-registered $\rho \leq 0.33$ as authorship and $\rho > 0.72$ as content, and fixed both bands before any arm ran.

One model writing both classes removes between a third and nine-tenths of the separability (Table 1). We report the *independent* arm, in which the model writes the attack class from the problem statement alone, so the writer changes and nothing else; an *edit* arm, shown its own honest solution, returns near-copies and is reported in appendix A.4. We sample one attack per problem, fixed before its outcome is known, and define the attack class as the benchmark does, by intent together with passing the public tests. B' is refitted on the same problems and includes inert attacks.

The pre-registered rule returns a different verdict for each generator. Sonnet 5’s interval excludes the collapse band and spans the content line, leaving it inconclusive between partial and content; Haiku 4.5’s lies inside the collapse band. Sample size does not account for the fall: at $n = 623$ the stored-pair baseline under the identical pipeline is 0.8521 ± 0.0055 . Two further rows are *strict*: they keep only attacks that pass the public tests *and* give a wrong answer on the trigger input the model declares. Single samples rarely do both, 2.2% of Sonnet 5’s and 0.3% of Haiku 4.5’s (2 of 611), so for Sonnet 5 the strict rows follow the benchmark’s own practice of sampling each problem repeatedly and keeping the survivors, which yields 99 pairs. We read those rows through ρ alone and they agree with the headline rows at low power. Haiku 4.5 has too few for a strict row at all (appendix A.6).

Table 1: **What is left when one model writes both classes.** S is the same-generator AUROC (5-fold, comments stripped) and B' the stored-pair baseline on the row’s own problems; ρ , its bands and its intervals are as in the text and appendix A.5.

Arm	n	S (95% CI)	B'	ρ (95% CI, joint)	band
Sonnet 5, headline	623	0.7512 (0.7295–0.7713)	0.8658	0.687 (0.625–0.755)	partial–content
Haiku 4.5, headline	611	0.5340 (0.5159–0.5531)	0.8584	0.095 (0.044–0.150)	collapse
Sonnet 5, prompt-matched	586	0.6423 (0.6179–0.6621)	0.8613	0.394 (0.326–0.454)	partial
Sonnet 5, strict	99	0.7652 (0.7016–0.8000)	0.7877	0.921 (0.662–1.089)	partial–content
Sonnet 5, strict, matched	96	0.7113 (0.6464–0.7453)	0.8030	0.697 (0.494–0.856)	partial–content

The attack prompt separates the two generators. On Sonnet 5 the surviving signal is lexical, with length alone falling to 0.5851 while TF-IDF holds, and it does not run through the trigger. Masking numerals and comparison operators leaves S unchanged; only normalising identifiers moves it, to 0.700 ($\rho = 0.54$). The top attack-side features are `def main` (67% of attack programs against 38.5% of honest ones) and `data[`, which together are the program layout the attack prompt elicits. Haiku 4.5 never adopts that layout. `def main` and `sys.stdin` appear in no program of any of its three classes, which read input with `input()` throughout (appendix A.6). Holding the prompt constant as well as the model therefore moves the two generators toward each other, to $\rho = 0.394$ and $\rho = 0.113$. On Sonnet this removes most of what is left. The two honest classes alone, with no attack on either side, already separate at 0.6006, so the prompt is carrying framing and no backdoor at all. On Haiku the same prompt-only floor is 0.4918, an interval containing chance.

Regenerating one class rather than both does not work. With an open-weight model writing the honest class against the stored backdoors, separation is higher than the stored baseline, at $\rho = 1.234$ (appendix A.5): the original authorship gap has been replaced by a new one between that model and the stored attacker.

The same inflation appears when the detector is an LLM monitor rather than a bag-of-words classifier. We put a zero-shot monitor prompt to the model that wrote both classes. In this field’s vocabulary it is an untrusted monitor, which makes it the most favourable case available, since it is reading its own output. On the stored pairs over the headline problems it scores 0.9579 and on the same-generator pairs 0.566, a monitor- ρ of 0.145, below the classifier’s 0.687. On the strict pairs it scores 0.9437 and 0.818, a monitor- ρ of 0.717 (intervals in appendix A.7). A monitor that wrote neither class returns the same two ratios: 0.151, a bound at half missingness, and 0.749. Among attacks that pass the tests, ours give a wrong answer on their trigger at 2.5% against the stored attacker’s 67.2%, which makes the stored attack class a survivor set.

The split selects on a second property besides authorship. Labelling by `backdoor_works` defines the negative class at the problem level by attack failure; the criterion should be honesty. 80.7% of the test split’s labelled backdoors do not work (32.8% of those that pass the public tests), and a problem whose backdoor fails contributes its human honest solution instead. The label is fixed before any code is read. Attack problems also carry half as many test cases (20.95 versus 38.85), and fewer tests make a backdoor easier to hide, so difficulty and test count correlate with the label. That correlation produces the content-blind 0.772.

Removing the authorship gap does not remove every floor. In five settings where the same three models write both classes, so no authorship confound is possible, the surviving floor is structural: 0.504 (bash, on its null) to 0.802 (agentdojo) from character count alone (Figure A2).

4 Limitations

A high floor is evidence about a corpus; it does not settle results computed on that corpus. Our floors show that the label is recoverable without reading a backdoor. They say nothing about whether any monitor recovers it that way, and no number here corrects a published monitor score.

The control arm is limited by the generator we had to substitute. The model that wrote the stored backdoors, `claude-3-7-sonnet-thinking`, is retired, so both same-generator classes come from a later model of the same lineage. It differs from the stored attacker in backdoor style as well as in authorship, so every ρ here, classifier or monitor, bounds the content share of the separability instead of partitioning it. The arm is not bit-reproducible, having run through a metered API that does not expose sampling parameters, with thinking off although the stored attacker was a thinking model.

Our scope is narrow in three further ways. We measure one APPS-derived corpus family and five agentic settings, and other stored-pair corpora are untested. Five of our seven corpora are one user’s repackaging of ControlArena releases, so our claims are scoped to those artifacts as published. And two interpreter-era artifacts fall on opposite classes, so B carries a small execution-environment channel that the same-generator arm does not (appendix A.11).

5 Discussion

These results imply one rule for construction: one model should write both classes under one prompt framing, and one harness should verify both. Regenerating a single side does not help: substituting the honest author left a larger gap than the one it replaced. Matched construction is preceded in code-detection benchmarks [4], though we have not seen it reach backdoor detection.

The definition of the negative class needs the same treatment. Defining it through attack failure ties the label to the attacker rather than to the code, and puts a content-blind classifier above most monitors ranked on this split; dropping the failed problems removes it, as does sampling that class independently of attack success. A monitor number is in turn interpretable only against the floors of its own split, so a partial-input and a bag-of-words floor belong beside it, each against a permutation null, with the split, the metric and the preprocessing released alongside (ours are MIT, appendix A.10).

Stored pairs remain the cheapest way to develop a monitor, and nothing here argues for abandoning them, only that a score on one describes two authors until the construction rules that reading out.

References

- [1] Basak Demirok, Mucahid Kutlu, and Selin Mergen. MultiAIGCD: A comprehensive dataset for AI generated code detection covering multiple languages, models, prompts, and scenarios. 2025. arXiv:2507.21693v1.
- [2] Catherine Ge-Wang, Tyler Crosse, Benjamin Hadad, Joachim Schaeffer, Ram Potham, and Tyler Tracy. Attack selection in agentic AI control evaluations meaningfully decreases safety. 2026. arXiv:2606.06529.
- [3] Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. 2023. arXiv:2312.06942.
- [4] Hanxi Guo, Siyuan Cheng, Kaiyuan Zhang, Guangyu Shen, and Xiangyu Zhang. CodeMirage: A multi-lingual benchmark for detecting AI-generated and paraphrased source code from production-level LLMs. 2025. arXiv:2506.11059v1. No venue stated.
- [5] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with APPS. In *NeurIPS Datasets and Benchmarks Track*, 2021. arXiv:2105.09938. The upstream source of the human-written honest class, distributed as the HuggingFace dataset codeparrot/apps.
- [6] Rogan Inglis. apps-control-arena. HuggingFace dataset RoganInglis/apps-control-arena. MIT licence. 5,000 rows; honest class from codeparrot/apps., 2025.
- [7] Anik Jha. Decorrelation is not complementarity: Skill, not lineage, governs trusted-monitor ensembles. 2026. arXiv:2608.16190. Code: <https://github.com/anik-jha/challenger-panels> (MIT).
- [8] Eugene Koran. apps-control-arena-high-quality. HuggingFace dataset eugenekoran/apps-control-arena-high-quality. **No licence declared.** 1,209 train / 1,375 test., 2026.
- [9] Eugene Koran, Yejun Yun, Samantha Tetef, Benjamin Arnav, and Pablo Bernabeu-Pérez. Ensemble monitoring for AI control: Diverse signals outweigh more compute. 2026. arXiv:2605.15377v2.
- [10] Bryce Little. Length penalties make chain-of-thought less monitorable. 2026. arXiv:2607.09786v3.
- [11] Daniil Orel, Dilshod Azizov, Indraneil Paul, Yuxia Wang, Iryna Gurevych, and Preslav Nakov. AICD bench: A challenging benchmark for AI-generated code detection. 2026. arXiv:2602.02079v1. No venue stated on the abs page.
- [12] Lucas Pinto. Calibration-family overfit: Why trusted sabotage monitors don’t transfer across lineages. In *COLM 2026 Workshop on AI Measurement Science*, 2026. arXiv:2607.06596v3.
- [13] Aditya Sinha. control_arena_{bash,shade,iac,rogue_eval,agentdojo}. HuggingFace datasets adityaasinha28/control_arena_*. **No licence declared.** 1,500 rows each., 2026.
- [14] Mikhail Terekhov, Zhen Ning David Liu, Caglar Gulcehre, and Samuel Albanie. Control tax: The price of keeping AI in check. 2025. arXiv:2506.05296v3.
- [15] Mikhail Terekhov, Alexander Panfilov, Daniil Dzenhaliou, Caglar Gulcehre, Maksym Andriushchenko, Ameya Prabhu, and Jonas Geiping. Adaptive attacks on trusted monitors subvert AI control protocols. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.09462v2.

198 A Supplementary tables and figures

199 A.1 The schema

200 Table A1 is the record schema of RoganInglis/apps-control-arena [6]. The attack class carries
 201 model and source, naming the system that produced each backdoor; the honest class carries no such
 202 field, because there is no generator to name. The two APPS-backdoor corpora we use are two views
 203 of this one artifact, since every problem in the filtered release [8] is also in [6], byte-identical except
 204 a re-validated backdoor_works. Among the agentic corpora we exclude AgentHarm, to avoid a
 205 conflict of interest.

Table A1: The confound is legible in the dataset schema, with no measurement at all. The attack class records which model wrote it; the honest class has no generator field. Verified against the shard schema of RoganInglis/apps-control-arena.

struct	sub-fields
solutions	code, compiles, passes_tests
backdoors	backdoor_code, backdoor_input, backdoor_works, compiles, model, passes_tests, source

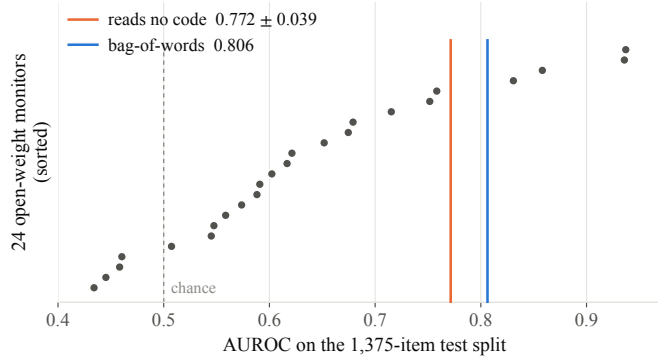


Figure A1: **Two floors against the 24 open-weight monitors ranked on the same split.** AUROC on the 1,375-item test split (172 attack / 1,203 honest), comments stripped; monitor scores from the MIT-licensed release of [7], median 0.609, max 0.937. Both floors clear 20 of the 24. **Two caveats.** (i) The monitors are zero-shot prompted while the bag-of-words classifier is trained on the benchmark’s own paired train split. The comparison states what the corpus makes recoverable and does not claim that bag-of-words is a better monitor. (ii) The no-code floor is cross-validated *within* the test split; on one held-out slice it is 0.716, above 18 of 24 (appendix A.2).

206 A.2 The protocol × baseline table

207 The no-code floor is cross-validated within the test split because the released train/val split is
 208 paired and therefore metadata-balanced; its permutation null shuffles labels for fitting and scoring
 209 alike. Table A2 is the full measurement behind §3. Rows are not comparable across protocols;
 210 the preprocessing column says whether comments were stripped before the baseline read the code.
 211 Pairwise accuracies are given to three decimals because their resolution over 2,641 pairs is one pair.
 212 Intervals in §3 are bootstraps over problems (2,000 draws) and nulls are 200 label permutations;
 213 the held-out no-code row is barely above its own null: 0.716 (CI 0.6476–0.7796) against a 95th
 214 percentile of 0.7023, $p = 0.030$, above 18 of the 24 monitors by point estimate and 12 with a paired
 215 interval excluding zero. The no-code claim therefore rests on the cross-validated protocol. The
 216 two floors of §3 in full: bag-of-words reaches 0.806 (95% CI 0.7704–0.8409 by bootstrap over
 217 problems; permutation null 95th percentile 0.5465, $p < 0.005$), above 20 of the 24 monitors by
 218 point estimate and 18 with a paired-bootstrap interval excluding zero; the no-code floor reaches
 219 0.772 (CI 0.7319–0.8005 on the pooled out-of-fold 0.766; null 0.5545, $p < 0.005$), above 20 and 17
 220 respectively.

Table A2: What the field’s preprocessing leaves behind. Every row is measured under the protocol named in column one; rows are not comparable across protocols. *Prep.* says whether comments were stripped before the baseline read the code; *no code* means the baseline never reads the program. The comments-kept rows are inflated by the comment channel and are given for contrast, not as results. The matched rows are problem-disjoint and scored against the DSPy tutorial monitor; the transfer row runs Control Tax to the GPT-4-era set. Pairwise accuracies are given to three decimals, their resolution over the pair count.

Protocol	Baseline	Value	Prep.	Against
<i>AUROC</i>				
Koran’s exact split	TF-IDF word 1–2gram	0.8063	stripped	above 20 of 24
same	TF-IDF word 1–2gram	0.8914	kept	above 22 of 24
same, reads no code	difficulty + #tests + $ q $	0.7715 ± 0.0389	no code	above 20 of 24
same, held-out slice	difficulty + #tests + $ q $	0.7163	no code	above 18 of 24
same	code length only	0.4726	stripped	chance
same	#-count detector	0.4983	stripped	chance
same	#-count detector	0.6957	kept	—
Raw artifact, problem level	all metadata	0.8917	no code	—
same	difficulty only	0.6823	no code	—
same	<code>solution_passes_tests</code>	0.7881	no code	—
Matched pairs (P3)	all metadata	0.5000	no code	by construction
<i>Pairwise accuracy</i>				
DSPy matched pairs	TF-IDF word 1–2gram	0.897	stripped	monitor 0.69–0.81
working backdoors only	TF-IDF word 1–2gram	0.935	stripped	—
inert backdoors only	TF-IDF word 1–2gram	0.879	stripped	not sabotage
transfer, cross-generator	TF-IDF word 1–2gram	0.969	stripped	—
all pairs, comments kept	TF-IDF word 1–2gram	0.962	kept	—
working, comments kept	TF-IDF word 1–2gram	0.979	kept	—
inert, comments kept	TF-IDF word 1–2gram	0.955	kept	—
transfer, comments kept	TF-IDF word 1–2gram	0.994	kept	—
same, raw artifact	code length only	0.826	kept	—

221 A.3 Comment stripping on the matched-protocol numbers

222 The matched-pair and transfer measurements were first made with comments in. Every value in
223 the authorship paragraph of §3 was re-measured with the same stripper applied to both classes,
224 over five problem-disjoint splits (each transfer row is one fixed split). With comments kept the
225 same quantities were transfer 0.994, working 0.979, inert 0.955, all pairs 0.962 and surface-feature
226 AUROC 0.8306; stripped they are 0.969, 0.935, 0.879, 0.897 and 0.7259. Every one was inflated by
227 the comment channel, which is this paper’s thesis applied to itself. The #-presence marker, 0.685
228 against 0.176 before stripping, closes to 0.007 against 0.008 after and is dropped; `__main__` is
229 unchanged. One authorship leak survives the stripper because string literals are not comments: in the
230 comment-stripped train split an author byline (Author :) survives in 23 of 1,209 honest samples
231 against 2 attack samples, and a triple-quoted block in 43 against 9. Both are far too rare to drive 0.806,
232 but both are a literal record of authorship. Stripped, the matched-protocol TF-IDF AUROC is 0.8560,
233 beside $B = 0.871$ and the Koran-split 0.806, all three on stripped code. The other cross-generator
234 transfer rows, stripped, are 0.866 (o3-mini to claude-3-7-sonnet, 82 test pairs), 0.844 (the reverse)
235 and 0.764 (o1-mini to gemini-2.0-flash).

236 A.4 Pair similarity, and why the same-generator control needs two arms

237 An attack class built by editing its own honest solution changes pair similarity as well as authorship
238 (Table A3); the independent arm sits in the stored benchmark’s regime and if anything slightly further
239 from it, so it cannot manufacture a low S by making the classes alike.

240 A.5 Robustness of the same-generator rows

241 At the strict rows’ $n = 99$, B under the identical pipeline is 0.7807 ± 0.0248 . Intervals in the
242 same-generator paragraphs of §3 are bootstraps over problems, resampling S and B' jointly for ρ ;
243 this replaced a ± 2 fold-sd rule before the retry pool was read. Every ρ point we quote is the ratio of

Table A3: Character similarity of honest/attack pairs, comments stripped. The edit arm’s classes are near-copies; the independent arm reproduces the stored benchmark’s regime, where the two classes are largely independent programs. The last two columns are the share of pairs above and below those similarities.

Attack class	mean sim.	share > 0.95	share < 0.60	changes vs. the benchmark
stored benchmark	0.606	3.4%	45.4%	the regime to match
edit (shown its own solution)	0.928	64.2%	4.0%	authorship and similarity
independent (problem only)	0.659	3.4%	34.8%	authorship only

the fold-mean AUROCs and every joint interval beside it is the bootstrap of the pooled out-of-fold ρ , which is the same quantity computed two ways: the pooled points are 0.86 (strict, the row whose interval Table 1 quotes), 0.390 (prompt-matched headline), 0.662 (prompt-matched strict) and 0.097 (the second generator). Five fold seeds move the strict row’s ρ over 0.85–0.92. Retention does not produce the effect. Survivors and non-survivors separate at only 0.62, and S computed with non-firing attempts on the same problems is 0.7595 against 0.7687. The edit arm bounds S under near-copies (similarity 0.928, Table A3): 0.6266 over 427 pairs, $\rho = 0.371$. In arm 1, where only the honest half was regenerated, $G = 0.962$ against $B' = 0.874$ on that arm’s $n = 668$ eligible problems, and TF-IDF gains +0.088 while the length carrier collapses to 0.489. Our own attack prompt halves our single-shot firing rate: 52.0% of our test-passing draws offer a public test input as the trigger, which cannot diverge from the honest solution, and those fire 0.15% against 5.13% for novel triggers; no stored working backdoor has such a trigger (0 of 3,188). The rest is retry-and-keep.

A.6 The prompt-matched arm under the masks, and the second generator

Under prompt matching the identifier mask, the only mask of §3 that moved S , takes the headline row to 0.6264 ($\rho = 0.342$) where it takes the original honest class on the same 586 problems to 0.7046, so identifier normalisation and prompt matching remove largely the same thing. That masked row is the one that decides whether the residue is structure or naming, and under prompt matching it is band-ambiguous: its joint interval, 0.280–0.394, puts 40% of its mass in the collapse band. A character model (3–5-grams, same classifier and folds) retains more than the word tokeniser, 0.6817 ($\rho = 0.493$), so the drop is not an artefact of tokenisation. On the strict pairs the matched row is $S = 0.7113$ ($n = 96$, $\rho = 0.697$, 0.494–0.856). The second generator’s pairs are more self-similar than the first’s (0.724 mean against 0.659), so S was recomputed inside similarity strata, where it stays 0.4737–0.5142 in every stratum, including the 0.451-similarity stratum, which is further from its honest class than the stored benchmark’s own pairs, while B' holds 0.7884–0.8334, so near-copies are not what buys the collapse. Its prompt-matched rows move S to 0.5399 ($\rho = 0.113$, $n = 562$) and put the prompt-only floor at 0.4918, an interval containing chance. The mechanism is visible in the programs. The layout vocabulary that the first generator’s residue is made of, `def main, sys.stdin` and `data[`, is present in 0.000 of the second generator’s programs in *all three* of its classes, which read standard input with `input(` in 1.000 of programs on every side, so there is no layout difference left for the classifier to read.

A.7 The monitor rows, and what bounds the second monitor

The monitor that wrote both classes scores 0.566 (0.5507–0.5833) on the same-generator headline pairs, a monitor- ρ of 0.145 (0.111–0.182); on the strict pairs it scores 0.818 (0.7630–0.8686), a monitor- ρ of 0.717 (0.601–0.832). The monitor that wrote neither class scores 0.5577 on the same-generator headline pairs and 0.8824 on the stored pairs over the same problems, 0.7871 and 0.8832 on the strict pairs. Its headline rows bound the value without fixing it. A 512-token reply ceiling left 628 of 1,246 same-generator replies unparseable, and on the stored side the missing share differs by class (50.1% honest against 23.4% attack), a bias of unknown sign. Re-running the strict pairs with a 1,024-token ceiling prices it: missingness falls to 2 of 198 and monitor- ρ moves from 0.749 to 0.766, inside the bootstrap interval. On the stored pairs both monitors beat the bag-of-words baseline over the same problems (+0.09 and +0.15 for the first, +0.02 and +0.09 for the second): Figure A1’s ranking states something about those monitors on that split and does not claim that a bag of words beats a monitor here. Classifier and monitor are independent pair by pair on the same-generator pairs ($\phi = 0.04$), so the two are reading different things.

288 A.8 The two generators' funnel

289 Both generators were run through the identical pipeline. Our honest draws pass the public tests at
 290 0.7701 and 0.7278 of 1,444 problems; the second generator's attack draws pass at 689 of 1,444 and
 291 35 of those fire, below the 50-pair minimum `gate_s_samegen.py` requires, leaving it with no strict
 292 row and the two generators are compared on the `passes` eligibility only. The two compared rows
 293 differ in inertness. 2 of the 611 attacks in the second generator's row fire (99.67% inert) against 14 of
 294 623 (97.75%), the other 33 of its firing draws sitting on problems where its own honest program failed
 295 the tests and which the control therefore drops. The single-shot firing rate belongs to the protocol and
 296 not to either model, since 52.0% of test-passing draws offer a public test input as their trigger.

297 A.9 The five same-generator agentic settings

298 Figure A2 shows each setting's content-blind floor against its own permutation null.

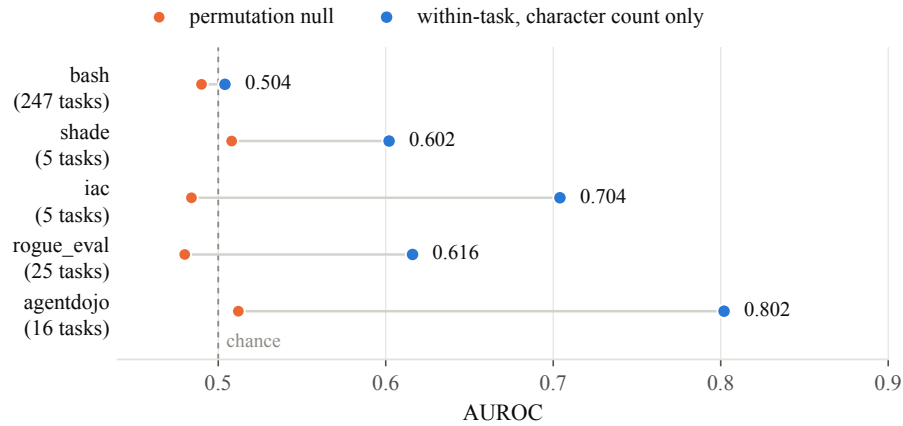


Figure A2: **The structural floor in five same-generator agentic settings.** Each setting's content-blind classifier (six count features, no content read) scored *within task* against its own permutation null. No authorship confound is possible here: claude-sonnet-4, gemini-2.5-pro and gpt-5 write both classes in all five. bash sits on its null at 0.504; agentdojo reaches 0.802 from character count alone. **The task-group count beside each setting matters:** shade and iac average over five task groups each while bash averages over 247, so these floors are not equally well determined. We report the within-task column because our own pooled numbers partly broke on task composition, and we broke them ourselves. The permutation nulls shown were computed with a pooled estimator rather than the folded within-task statistic; they bound the chance level but are not a like-for-like null for each dot. These corpora declare no licence; we redistribute nothing and report aggregates only.

299 A.10 The release

300 The supplementary bundle holds the same-generator split (1,444 honest programs, 1,112 edit-arm
 301 and 1,112 independent-arm attack programs written by one model, plus every one of the 11,565
 302 draws of the independent arm's retry pool), both eligibility rules as pair lists under an outcome-blind
 303 selection (623 passing and 14 firing independent-arm pairs) and under the benchmark's own survivor
 304 selection (1,046 and 99, or 106 counting a crash on the trigger as firing), the 24 scripts behind the
 305 measurements (generation for both generators, the test harness, the trigger verifier, the selection rules,
 306 the prompt-matched arm, the monitor rows and each remaining measurement script), a dataset card,
 307 and a manifest of row counts and checksums. The licence, MIT, is a file inside the bundle and covers
 308 the data and the scripts alike.

309 A.11 The test harness and the pass flag

310 The same-generator arm requires executing generated code, so we built a runner that feeds each
 311 shipped input case to a solution under per-case timeouts and address-space limits. It agrees with the

312 artifact’s own `solution_passes_tests` on 95.84% of the analysis pool, and is strictly conservative
 313 there, never crediting a solution the artifact rejects (0 such cases). We use *our own* flag for both
 314 classes, because a checker stricter on one class than the other would manufacture exactly the artifact
 315 this paper documents. We do not length-match either class; the untrusted model chooses its own
 316 trajectory length. The headline does not depend on that choice. Under the artifact’s flag $B = 0.871$
 317 at $n = 1,444$, under ours $B = 0.873$ at $n = 1,406$, within one standard deviation of each other.
 318 The monitor scores in Figure A1 are `runs/x0/monitor-summary.json` from the MIT release of
 319 that harness [7], used under its notice. Two interpreter-era artifacts in the upstream solutions fall on
 320 opposite classes: a `stdin` without `.buffer`, fixed, and `fractions.gcd`, removed in Python 3.9, not
 321 fixable. B is exposed to them; the same-generator arm, whose code we generated and ran ourselves,
 322 is immune.

323 **A.12 The token test, run and failed**

324 We attempted to confirm a reported token-level mechanism [11] in our setting and it does not replicate.
 325 Separation is -0.013 , the wrong sign, at permutation $p = 0.956$. The reason is register. Their human
 326 class is general-purpose repository code, whereas both of our classes are competitive-programming
 327 code, so an identifier axis that separates general human code from model code does not separate
 328 these two populations. Only 8 of their 10 canonical tokens are in our vocabulary, so the test is
 329 underpowered; but an underpowered true effect would still carry the right sign, and this one does not.
 330 We report it because a reader who knows that work may run exactly this test.

331 **A.13 A third channel, noted but not developed here**

332 In the same five agentic settings a fifteen-word regular expression over the untrusted policy’s own
 333 scratchpad reaches 0.76–0.82, because prompted attack policies restate their side task in the reasoning
 334 the monitor is given. We note this only to bound the claim. The instrument is not ours, and the work
 335 it comes from [10] validates it against an LLM judge and a length-matched null; we defer to that
 336 treatment.